FraudDetectAI

Somyadeep Tarat (24BKT0104)
Aarav Gupta (24BCE0163)

A DBMS project with an evidence-first fraud detection extension
Proposed novelty: Funding Ambiguity Bounds Certificate (FABC)
Academic design proposal, 12 October 2026

FABC is a proposed extension. This report does not claim it has been implemented, benchmarked or proven patentable. The description of the existing system is taken from the original proposal's review of the repository; it was not re-checked here.

1. Summary

Customers own bank accounts and registered devices. A FastAPI service validates internal transfers, locks the affected rows, changes balances, runs a fitted fraud model and saves a prediction. PostgreSQL creates alerts and redacted audit records through triggers. Streamlit displays banking operations and investigation views. A separate PaySim pipeline trains transaction-only and graph-enhanced models.

FABC calculates how much of an outgoing payment must, and how much could, be funded by recent incoming transfers. It stores the answer together with the exact ledger evidence needed to replay it. Fast receive-and-forward behaviour can indicate a pass-through account, but it also occurs in legitimate collections. Money in an account is fungible, so timing alone cannot establish which receipt funded a payment. FABC states that uncertainty explicitly and turns the explanation into a reproducible database record.

2. Novelty check

2.1 Verdict

The novelty is partial and narrower than the original proposal implied. The interval mathematics itself is not new. What may be new is the combination: (a) storing the exact feasible interval, instead of a single attribution, as the review quantity in a banking DBMS; (b) binding it to a validated, immutable ledger prefix and committing it atomically with the transfer so it can be replayed; and (c) measuring it against FIFO, haircut and velocity baselines. I found no source in a limited web search that combines all three. That is not evidence of novelty, only the absence of an obvious match.

2.2 What is already known

- Single-point tracing rules for mixed funds are standard. Blockchain forensics uses Poison, Haircut and FIFO taint methods (Moser, Bohme and Breuker; Anderson et al., arXiv 1901.01769; Tironsakkul et al., arXiv 1906.05754). LIFO is also described.
- Trust law solved the same fungibility problem long before. Clayton's Case (1816) applies FIFO. Re Hallett's Estate (1880) presumes a trustee spends his own money first. The lowest intermediate balance rule (Roscoe v Winder [1915] 1 Ch 62; applied by the Privy Council in Brazil v Durant) caps what can be traced at the lowest balance reached. Forensic accountants use it in fraud cases.
- In FABC terms, the upper-bound path (old money spent first, recent money kept) is the same logic as Re Hallett and the lowest intermediate balance rule. The lower-bound path (recent money spent first at earlier debits, old money first at the target) is its mirror image. Taking both extremes of a mixed-account balance is therefore a natural idea, not an unexplored one.
- Every single-point rule (FIFO, LIFO, haircut) produces an allocation that is feasible, so its result must lie inside [lower, upper]. Poison can exceed the upper bound. This is useful as a correctness test, not as a novelty claim.
- Pass-through and layering detection by timing and amount matching is established (FlowScope, AAAI 2020; a recent arXiv paper on pass-through templates, 2609.20737).

- Funds-flow graph tracing for payments is already patented. Stripe's transaction tracing family (for example US11514534 and US20240119538A1) maps funds flows into and out of accounts and traces contributing transactions. Hash-chained, verifiable ledger tables in databases also exist (for example SQL Server ledger tables); this is from general knowledge and was not searched.

2.3 Where novelty may remain

- Reporting an exact attainable interval plus an ambiguity width as the stored analyst evidence, so the system never claims a unique funds path.
- The transaction protocol: a per-account sequence, a validated contiguous prefix, full evidence membership, and commit of certificate and transfer in one transaction, with INCOMPLETE rather than a false bound when history is truncated.
- The measured effect: whether a lower-bound flag gives better review quality, or equal quality with a more faithful explanation, than FIFO, haircut or velocity rules on benign counterexamples such as payroll, merchant settlement and family pooling.

2.4 How to strengthen the claim

- Extend from one account to several accounts using shared-capacity (flow-conservation) constraints, so interval bounds hold jointly across hops. This is harder than the MVP and is more likely to be distinct from single-account taint rules.
- Add incremental bounds with exact window-expiry logic and prove replay equivalence with the full scan.
- Show a technical effect that is measured, such as lower evidence storage, bounded query cost or consistency under concurrent writes, rather than asserting it.
- Do the experiments in section 12. A negative or modest result is still a valid academic outcome.

2.5 Limits of this check

This was a short web search of papers, legal sources and patent listings. It is not a patent search, it did not compare claims or priority dates, and it is not a patentability opinion. I could not confirm the specific number, date or claims of US11854091B1 cited in the original draft; Stripe's family above is the confirmed related disclosure. Section 3(k) of the Indian Patents Act excludes mathematical methods, business methods, computer programmes per se and algorithms, and the CGPDTM CRI Guidelines 2025 apply, so the formula, hash, lock or dashboard alone should not be presented as an invention. Before any filing decision, search patent families and papers for funding-source bounds, lower and upper taint bounds, provenance snapshots and ledger certificates, check Indian and international equivalents, and consult the institution IP cell or a patent professional. Do not publish details before that review.

3. Existing implementation

- Interface: Streamlit banking, DBMS demonstration and original ML analysis pages.
- API: FastAPI routes and validated request schemas in app/api and app/schemas.py.
- Transfer service: app/services/transactions.py checks ownership, status, balance and currency, takes ordered account locks and persists atomically.
- Online detection: fitted PaySim baseline through app/services/inference.py, plus separate SQL behaviour features and a heuristic risk.
- Database: SQLAlchemy entities, Alembic migrations, PostgreSQL triggers, audit logs, views, indexes and functions. SQLite is a local fallback.
- Offline ML: src/fraudgraph with validation, feature engineering, NetworkX graph, LR / RF / XGBoost, evaluation and explanations.

One transfer has one atomic outcome. The service locks the initiating customer and both accounts, writes the debit and credit snapshots, queries historical behaviour and runs inference. It stores raw model probability and final risk, and prediction insertion invokes the alert trigger. Essential failures roll back balances, transaction, prediction, alerts and audits together.

Current risk = max(model_probability, 0.9 x behaviour_score). This is an alert heuristic, not a calibrated probability. FRAUD and SUSPICIOUS are workflow labels, not established facts about criminal activity. A FLAGGED transfer still posts money; the application does not implement a payment hold.

Limits that matter to the proposal

- Only same-currency internal transfers are supported.
- The live API uses a baseline model, not the offline graph artifact.
- Full-graph aggregates and fraud-rate features can leak future information or labels.
- PaySim training is not validation on real bank customers.
- SQLite cannot demonstrate PostgreSQL row-level locks or stored functions.

4. Proposed contribution and financial reasoning

Research question: can a database produce an exact, replayable interval for the recent-money share of a transfer, and use a high unavoidable share to prioritise review while controlling false alerts and transaction latency?

- Fungibility: an account balance does not label individual rupees by origin. Several allocations can explain the same ledger.
- Recent funding: credits recorded within a chosen window before the target debit. Recent does not mean illegal.
- Lower bound: the least amount of the payment that must have come from recent credits under every feasible allocation. It is reached when earlier debits consume recent money first and the target debit consumes old money first.
- Upper bound: the most that could have come from recent credits under some feasible allocation. It is reached when old money is spent first.
- Ambiguity width: (upper - lower) / payment amount. A wide interval weakens attribution claims.
- Certificate: a stored result, policy version, ledger sequence boundary and ordered evidence, committed with the transfer.

The minimum version needs three tables, a per-account sequence counter, one indexed ordered history query and an O(n) Decimal algorithm. It uses the existing FastAPI, SQLAlchemy, Alembic, PostgreSQL and Streamlit stack. No GNN, blockchain or new distributed service is needed. ML probability stays separate; the analyst sees the lower bound, upper bound and data-completeness status.

5. System architecture

The service is Streamlit, then FastAPI plus the transfer service, then PostgreSQL. Inside the atomic transfer boundary the existing path is: validation and ordered row locks, debit and credit with the transaction row, model inference and stored prediction, then alert and audit triggers, ending in commit or full rollback. The batch path is separate: PaySim to features and graph, then training, then artifacts.

Proposed FABC runs inside the same boundary, after the locks and before final persistence. It reads ordered entries from the source account ledger, computes the recent-funding bounds and saves an evidence certificate alongside the prediction. The existing alert and audit mechanism continues to operate. All essential writes commit together, and a FLAGGED transfer still moves funds.

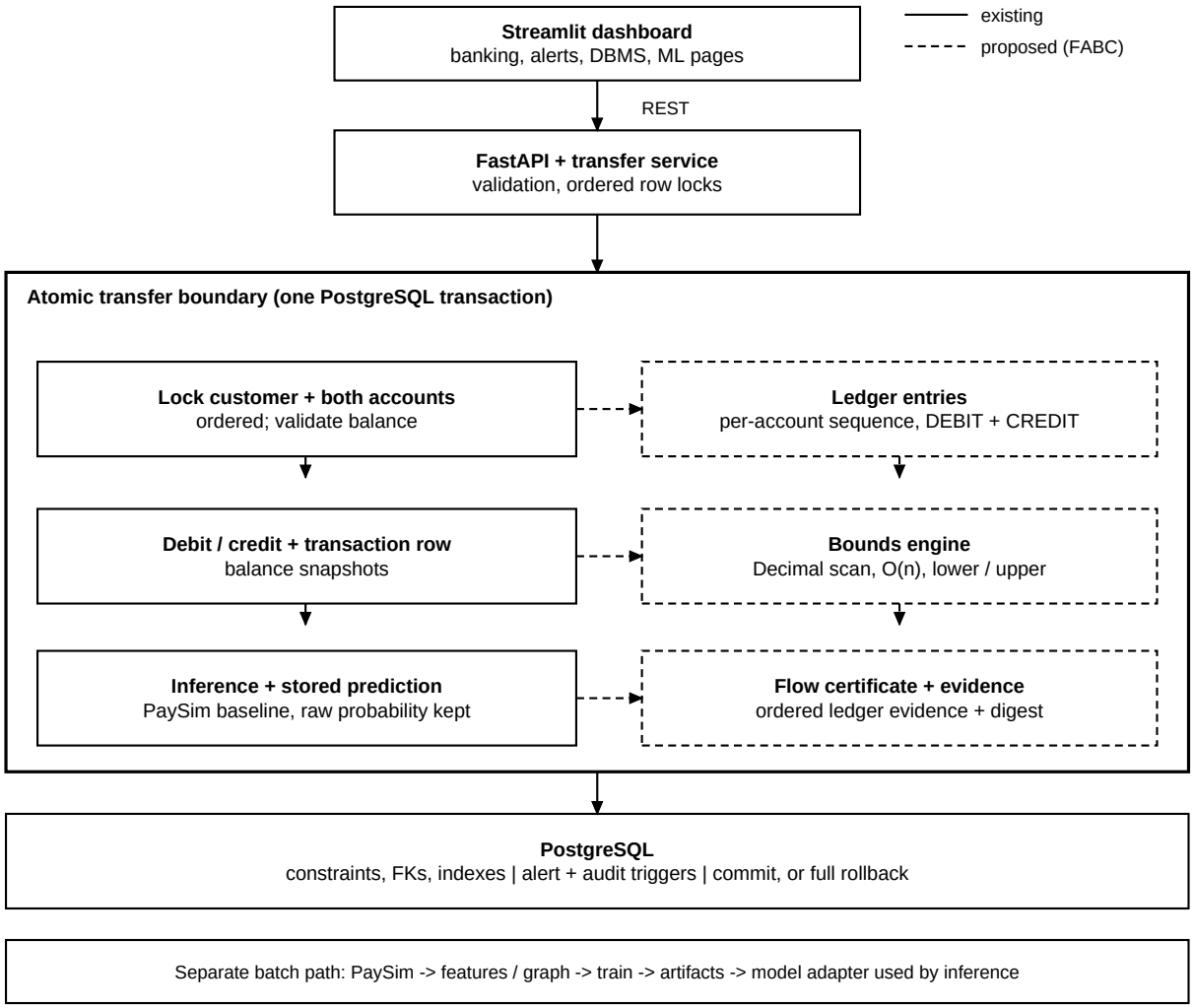


Figure 1. System architecture. Solid boxes exist today; dashed boxes are the proposed FABC layer inside the same transaction.

6. Existing database relationships (app/db/models.py)

```
CUSTOMERS (PK customer_id, UK email)
  1:0..N ACCOUNTS (PK account_id, FK customer_id, balance, currency)
  1:0..N DEVICES (PK device_id, FK customer_id, UK device_identifier)
ACCOUNTS 1:0..N TRANSACTIONS as source, and 1:0..N as destination
DEVICES 0..1:0..N TRANSACTIONS (device is optional)
TRANSACTIONS (PK transaction_id; FKs source, destination, customer, optional device;
  amount and balance snapshots)
  1:0..1 FRAUD_PREDICTIONS (PK prediction_id, FK and UK transaction_id)
  1:0..N FRAUD_ALERTS (PK alert_id, FK transaction_id, FK and UK prediction_id)
AUDIT_LOGS (PK audit_id; table_name + record_id; no polymorphic FK)
RISK_SETTINGS (PK settings_id = 1; fraud_threshold; policy, no entity FK)
```

The schema allows at most one prediction per transaction and at most one alert per prediction. Exactly one prediction on a completed service transfer is an application invariant. Composite FKs enforce source-account ownership, device ownership and agreement between an alert and its prediction and transaction.

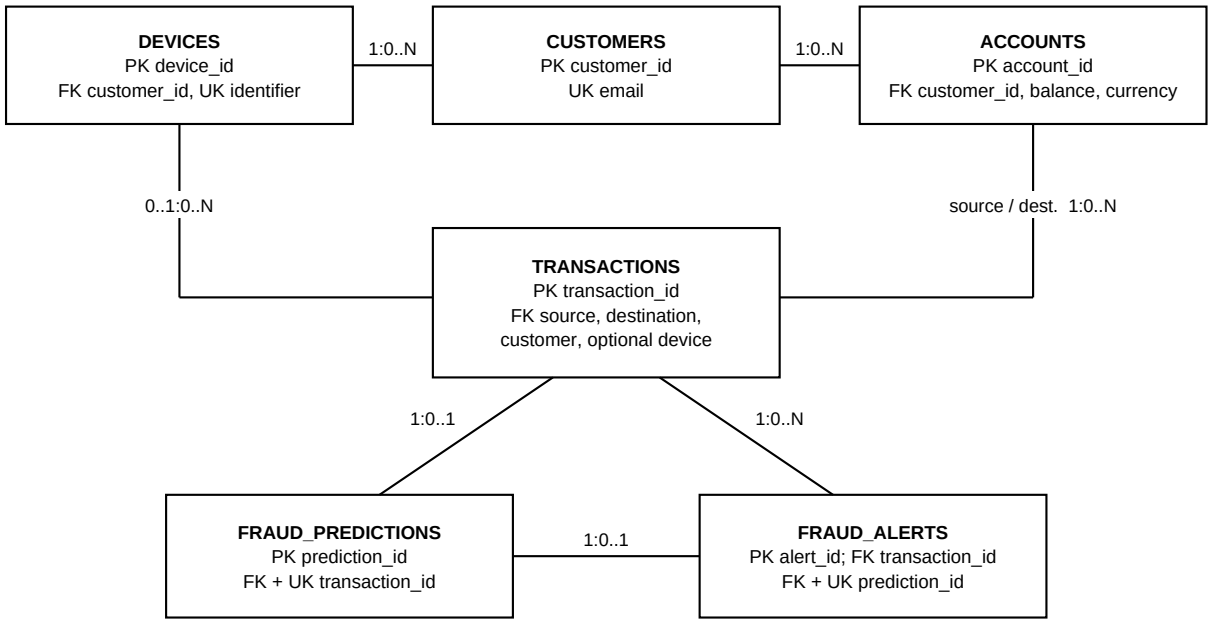


Figure 2. Existing entity relationships (parent : child cardinality).

7. Proposed schema

- accounts.ledger_seq: a BIGINT counter advanced under the account lock. It orders every credit and debit on that account; it is not a global commit sequence.
- ledger_entries (new): PK (account_id, seq); FK to account and transfer; UNIQUE (transaction_id, side); side is DEBIT or CREDIT; recorded_at, payload_hash; positive NUMERIC amount, nonnegative balances, before / after arithmetic checks. Exactly two entries per transfer is a deferred check or service invariant.
- flow_certificates (new): PK certificate_id; unique transaction FK; source account; opening sequence and balance; target debit sequence; window bounds; lower and upper amounts; status; evidence digest; algorithm and policy versions.
- certificate_evidence (new): PK (certificate_id, account_id, seq); FK to certificate; composite FK (account_id, seq) to the ledger entry; ordinal and role (OPENING_ANCHOR, HISTORY or TARGET); UNIQUE (certificate_id, ordinal). Store every required entry, not just selected suspicious receipts.

For a valid certificate enforce 0 <= lower <= upper <= target amount. A composite FK to a UNIQUE (transaction_id, account_id, side, seq) ledger key can bind the target to the actual source DEBIT. Cross-row conservation and two-entry completeness need deferred constraint triggers or a restricted write service; a row CHECK alone is insufficient. This proposal installs no schema objects.

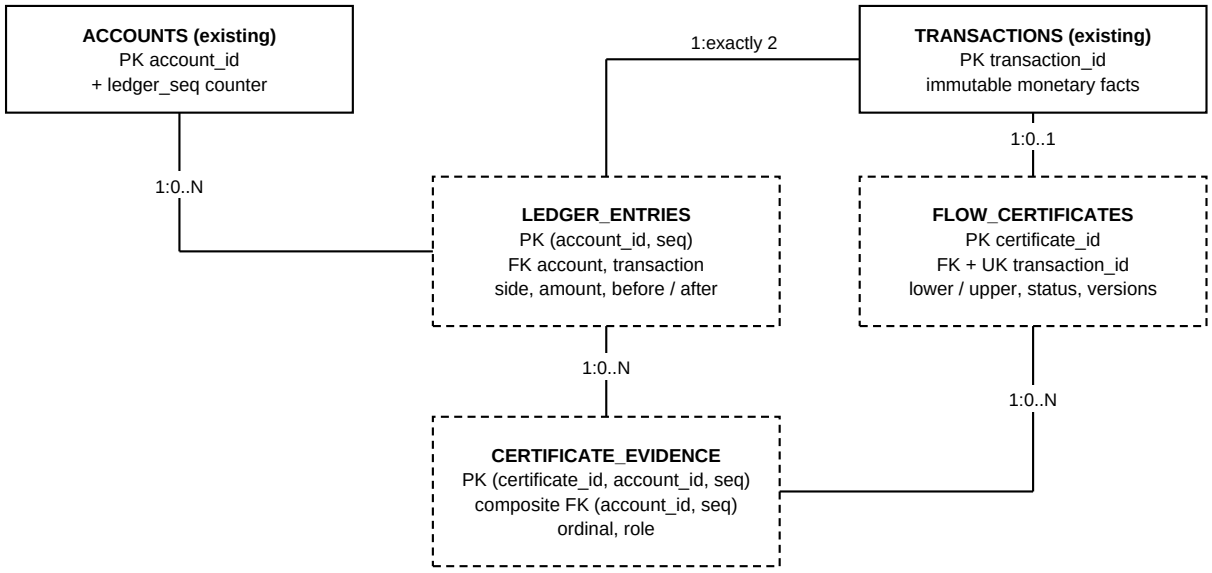


Figure 3. Proposed tables (dashed) and their links to existing tables (solid).

8. The exact bounds algorithm

Scope: one account, one fixed window, all posted movements. Let x be the proposed debit and B the balance at the start of the window. The opening balance is treated as old money, even if its origin is unknown. Read all account entries after the window boundary and before the target debit; all credits in this interval are recent. No overdrafts, unrecorded balance changes, fees, FX or missing movements are allowed. Track B exactly in Decimal or integer minor units. Sequence order governs causality; the server-recorded window determines which credits qualify.

L and U are the minimum and maximum recent-money balance that could remain. Start with L = U = 0.

```
For a credit c:
    B = B + c
    L = L + c
    U = U + c

For an earlier debit d (require 0 < d <= B):
    L = max(0, L - d)
    U = min(U, B - d)
    B = B - d

Before the target debit x (require 0 < x <= B):
    lower_amount = max(0, x - (B - L))
    upper_amount = min(x, U)
    lower_share  = lower_amount / x
    upper_share  = upper_amount / x
```

Why the bounds are exact under these assumptions. At a debit, spending recent funds first minimises the recent balance left, and spending old funds first maximises it. Both choices are feasible because funds are fungible and balances stay nonnegative. Each update is monotone in the previous recent balance, so the extremes carry through by induction over credits and debits. At the target, at most B - L old money can be available in the most recent-heavy case, and at most U recent money can be available, which gives the two bounds. The arithmetic of both worked examples below was re-checked and is correct.

The scan takes O(n) arithmetic operations with O(1) working state; keeping n evidence rows costs O(n) storage, and a B-tree prefix lookup adds query overhead. Individual payment intervals are marginal bounds: their upper amounts must not be summed as if jointly attainable. Multi-hop attribution needs additional shared-capacity constraints and is future work.

9. Worked examples and review policy

Example A: recent money is unavoidable

Event	Balance B	Recent balance [L, U]
Window opens with INR 1,000	1,000	[0, 0]
Credit 4,000 from A	5,000	[4,000, 4,000]
Credit 3,000 from B	8,000	[7,000, 7,000]
Credit 2,000 from C	10,000	[9,000, 9,000]
Earlier debit 2,000	8,000	[7,000, 8,000]
Target debit 7,500	(B = 8,000)	lower = 7,500 - (8,000 - 7,000) = 6,500
		upper = min(7,500, 8,000) = 7,500

Lower share = 6,500 / 7,500 = 86.67%; upper share = 100%. At least INR 6,500 must be recently received money under every feasible allocation. Three incoming counterparties plus a fast onward payment make this a candidate for pass-through review.

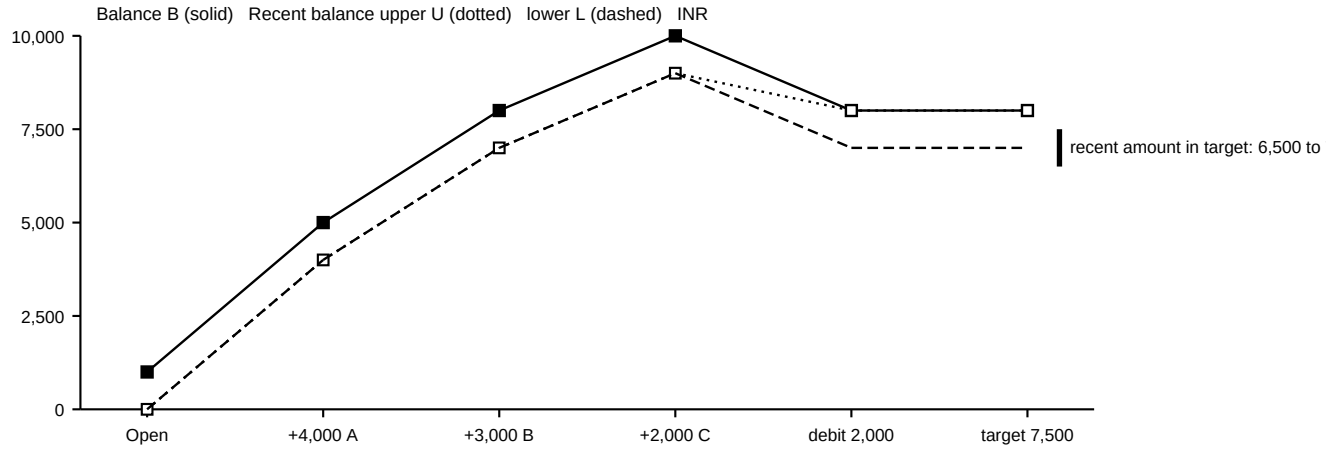


Figure 4. Example A: balance and the possible range of recent money after each event.

Example B: a funded business account stays ambiguous

Opening balance INR 100,000, recent receipts INR 9,000, target debit INR 7,500. The interval is [0, 7,500], or [0%, 100%]. The account could pay entirely from older funds. A heuristic based only on nearby receipts might imply a linkage the ledger cannot establish.

Initial review policy (not a probability)

For a course demo, request review when lower_share >= 0.80 AND at least three distinct incoming counterparties appear in a ten-minute window. These are starting parameters for evaluation, not regulatory thresholds. Store the flow result separately from model_probability and behaviour_score. A lower bound of zero does not prove innocence, and a high lower bound does not prove fraud: merchant settlements, family collections and payroll accounts can show the same pattern. If evidence is incomplete, status is INCOMPLETE and bounds are NULL, not zero. A flow flag queues investigation; it does not silently block the transfer.

10. SQL and transaction protocol

```
SELECT account_id, seq, transaction_id, side, amount,
       balance_before, balance_after, recorded_at
FROM ledger_entries
WHERE account_id = :source_account
  AND seq > :opening_seq
  AND seq < :target_debit_seq
ORDER BY seq;
```

```
-- Proposed supporting index:
CREATE INDEX idx_ledger_account_recorded_seq
  ON ledger_entries (account_id, recorded_at, seq);
```

Find the first entry at or after window_start using the same account prefix. The preceding entry is the opening anchor; its balance_after is the opening balance. If no movement occurs in the window, use the locked pre-transfer balance and the preceding sequence. Validate sequence continuity and every before / after snapshot. A missing anchor or balance gap makes the result INCOMPLETE.

Commit protocol

- 1. Begin a fresh database transaction. Keep the customer lock and globally ordered account locks. All movement writers must acquire the affected account locks before changing balances or allocating sequences.
- 2. Allocate one new sequence on each account. Choose server recorded_at after acquiring locks and enforce nondecreasing timestamps per account. Store exactly one DEBIT and one CREDIT per internal transfer. Caller-supplied event timestamps are metadata, not ordering keys.
- 3. Query the source prefix ending immediately before the new DEBIT. Compute bounds; save the certificate, opening anchor, full evidence membership and target reference. Preserve the ordinary inference result and version the flow policy.
- 4. Save the prediction and any review flag. Triggers write audits and alerts; every essential write commits together, and a failure aborts all writes. A bounded scan cap returns INCOMPLETE with a reason and no numerical claim; it never computes a false exact bound from truncated history.

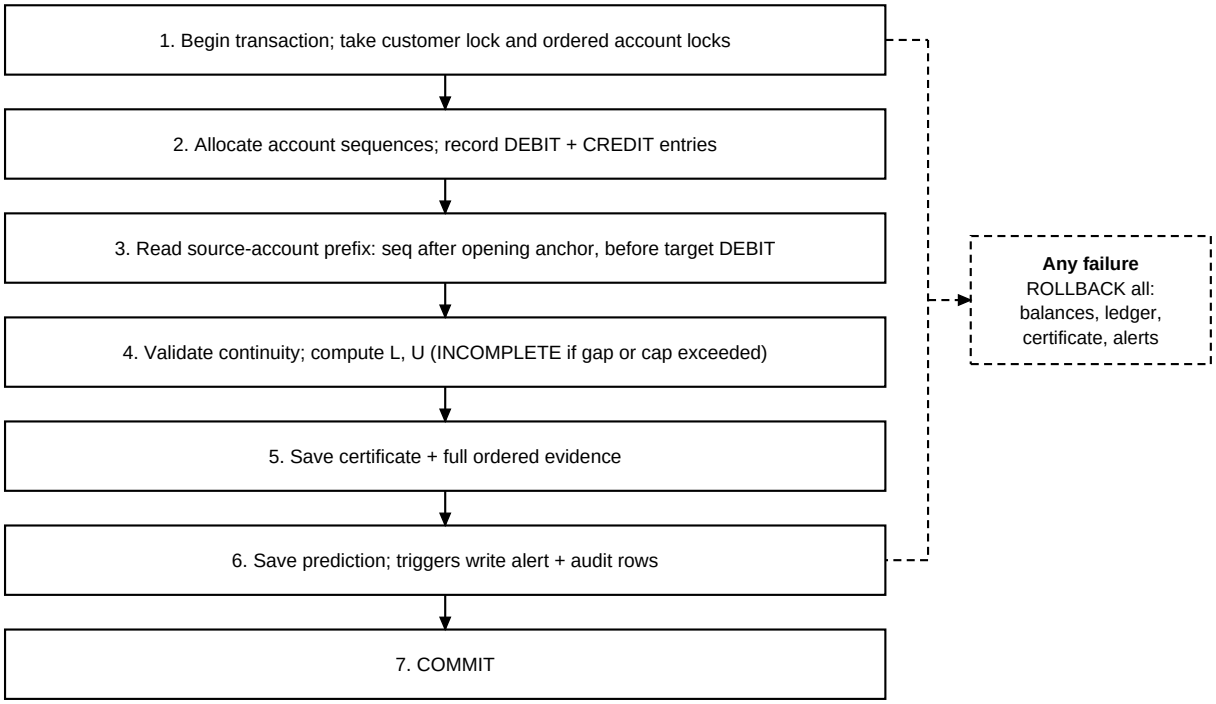


Figure 5. Commit protocol. Any failure rolls back every write.

Current locking is adequate only if every balance movement obeys the account-lock protocol, so direct SQL writers must be restricted. A separate multi-account graph scan would require a stable snapshot or serializable protocol and retries; sequences alone do not create a global snapshot (PostgreSQL, Transaction Isolation).

11. DBMS concepts to demonstrate

- Primary and foreign keys: the customer, account, device, transfer, prediction and alert chain; proposed composite ledger references stop evidence pointing at a nonexistent entry.
- Constraints: nonnegative balances, positive amounts, distinct accounts, ownership FKs, monetary arithmetic and score ranges. Try an invalid insert and show rejection.
- Normalisation: separate customer, account, device, prediction and alert facts. Certificates separate a derived result from a many-to-many evidence relation.
- Denormalisation: transaction.customer_id and alert.transaction_id are deliberate redundant navigation keys, with composite FKs preserving agreement. Do not claim strict 3NF for every table.
- Atomicity: inject an inference or certificate write failure and show that all balances and associated rows roll back.
- Isolation: run opposite-direction or simultaneous debits; show ordered row locks and no double spending.
- Consistency and durability: balance checks and ledger continuity preserve valid state; committed data survives reconnect or restart.
- Triggers and procedures: existing alert and audit triggers and PostgreSQL velocity functions; a proposed deferred check for exactly two ledger entries.
- Indexes and query plans: existing customer-time and risk indexes; the proposed account-time-sequence index. Use EXPLAIN ANALYZE at meaningful volumes.
- Views and joins: existing suspicious-transaction and customer-risk views; a proposed certificate detail view joining results and evidence.

Reproducibility and retention

Hash the canonical ordered evidence plus the opening balance, target, window and algorithm / policy version, using fixed decimal strings, UTC timestamps and a specified field order. A digest detects mismatched replay inputs; it is not proof against a privileged administrator who can rewrite data and hashes, so append-only permissions and backups remain necessary. Keep ledger monetary facts immutable; later alert reviews are separate workflow events. Existing demo cleanup deletes disposable rows, so the ledger needs a dedicated demo dataset or explicit retention handling. Never silently cascade-delete evidence used by a certificate.

12. Implementation roadmap and evaluation

Roadmap (about 2-3 weeks for a small team; a planning assumption)

- Days 1-3, schema and ordering: new Alembic migration, ledger counter, append-only entries, opening anchors, indexes, reproducible seed histories. Existing migrations stay untouched.
- Days 4-6, bounds engine: app/services/flow_bounds.py with Decimal math, contiguous-prefix validation and clear statuses. Verify against exhaustive small integer allocations.
- Days 7-10, atomic integration: extend process_transfer to write ledger and certificate in the same transaction; add idempotency keys; injection and concurrency checks.
- Days 11-13, analyst interface: lower and upper shares, opening balance, chronological evidence, policy version, a separate flow review flag and a replay endpoint.
- Days 14-18, evaluation and viva: baselines, ablations, query-plan measurements, rollback and replay rehearsal.

Minimum viable scope: one account, one ten-minute window, internal transfers only, exact interval scan and persisted evidence. Start with a separate flow flag on the certificate. If integrating into fraud_alerts, add a new versioned trigger policy, keep the one-alert-per-prediction constraint and merge reasons rather than inserting duplicate alerts. Extend app/services/transactions.py after locks and before commit; use a separate synthetic evaluation script with saved seeds; do not change the baseline model feature contract. A

fixed-window scan is easier to verify than a cache whose boundary expires continuously, so measure the scan first, set an explicit history cap and surface INCOMPLETE when exceeded.

Evaluation

Generate balance-valid internal ledgers with planted pass-through patterns and matched benign patterns: merchant aggregation, salaries, family pooling, refunds, long-lived balances and split payments. Preserve seeds and withheld labels. These test designed scenarios, not real-bank generalisation. PaySim can benchmark the existing transaction model but has an hourly step, so it cannot establish ten-minute timing, and it lacks the local ledger protocol and devices. Use chronological train / validation / test splits, select windows and thresholds on validation only, exclude target and future graph aggregates at inference, and compare against a repaired historical graph baseline rather than the potentially leaked full graph.

- Correctness: exhaustive allocation oracle on small integer histories; attainability of both bounds; boundary times, prior debits, gaps, opening anchors, zero history; check that FIFO, LIFO and haircut results fall inside the interval.
- Detection baselines: current ML plus behaviour; velocity plus fan-in heuristic; FIFO attribution; FABC lower-bound flag. Same data and review budget.
- Ablations: upper-bound-only versus lower-bound flag; no counterparty condition; varied opening balance and window; with and without persisted certificate.
- Detection metrics: precision, recall, average precision, false alerts per 1,000 benign transfers, recall at a fixed review budget, alerted amount.
- Database metrics: p50 / p95 / p99 transfer latency, scanned rows, certificate bytes, lock wait, throughput, rollback failures, replay agreement.
- Concurrency and recovery: simultaneous credits and debits, failed inference, certificate failure, retry and idempotency, reversal as a new movement, restart persistence.

No improvement figures are asserted. A successful outcome may be a more faithful explanation at similar detection quality; measure the cost of that explanation as well as its benefit.

13. References and viva explanation

- [1] Li et al. FlowScope: Spotting Money Laundering Based on Graphs. AAAI 2020, 34(4), 4731-4738. DOI 10.1609/aaai.v34i04.5906.
- [2] Stripe. Systems and methods for transaction tracing. US11514534 and US20240119538A1. The original draft also cited US11854091B1 (published 26 December 2023); its number and claims were not confirmed here and must be checked before citing.
- [3] Intellectual Property India. Patents Act, Section 3(k).
- [4] CGPDTM. Guidelines for Examination of Computer Related Inventions (CRIs), 2025.
- [5] PostgreSQL documentation. Transaction Isolation.
- [6] Anderson et al. Tendrils of Crime: Visualizing the Diffusion of Stolen Bitcoins. arXiv 1901.01769.
- [7] Tironsakkul et al. Probing the Mystery of Cryptocurrency Theft: An Investigation into Methods for Taint Analysis. arXiv 1906.05754.
- [8] James Roscoe (Bolton) Ltd v Winder [1915] 1 Ch 62; Federal Republic of Brazil v Durant International Corp (Privy Council, JCPC 2013/0069).

Repository evidence reviewed in the original proposal: README.md; docs/ARCHITECTURE.md; docs/DATABASE_DESIGN.md; app/db/models.py; app/services/transactions.py; app/services/features.py; src/fraudgraph/models/train_graph_enhanced.py.

A 60-second project explanation

"Our project is a relational banking system with AI-assisted fraud review. Transfers, predictions, alerts and audit history commit together, and constraints protect account ownership and balances. Our proposed addition asks how much of a payment must come from recent receipts. Because money is mixed, we compute an interval rather than assert a single source. We save that interval with the exact ordered ledger evidence so a reviewer can reproduce it. Known tracing rules such as FIFO and haircut give single answers inside that interval; we test whether the interval is a better basis for review. We will test correctness, benign counterexamples, review quality and database overhead. This is a research proposal; we do not claim a patent or proven detection improvement."

Strongest demonstration

Run the two numerical examples, show a saved bounds certificate and replay, inject a write failure and show complete rollback, then run concurrent transfers and verify that every certificate refers to a valid source-account prefix. Label these as future acceptance demonstrations until the extension is built.